

for-Loops (0): Basics von for-Schleifen

- Iterieren/Durchlaufen von Sequenzen (z. B. Listen)
- **FÜR** jedes Element **IN** Sequenz:
Führe Anweisungen aus
- **Python-Syntax für for-Loops (Schleifen):**

```
for item in list_of_items:  
    print('This will be repeated.')
```
- Laufvariablen können beliebigen Namen haben
- Wiederholter Code-Block mit beliebig vielen Anweisungen

for-Loops (1): Beispiel

```
lectures = ['Maths', 'English', 'Computer Science']  
  
for lecture in lectures:  
    print(f'I love {lecture}')
```

for-Loops (2): Beispiel

```
lectures = ['Maths', 'English', 'Computer Science']
```

```
for lecture in lectures:
```

```
    print(f'I love {lecture}')
```

Legende

Iterierte Sequenz (z. B. Liste)

Definition der Sequenz

for-Loop Definition (Syntax)

Wiederholter Code-Block

Lauf-/Schleifen-Variable

while Loops (0): Basics

- Ausführen von Anweisungen bis Bedingung nicht mehr erfüllt
 - „Solange Bedingung erfüllt, führe Anweisungen aus“
- **SOLANGE** *Bedingung erfüllt:*
Führe Anweisungen aus
- **Python-Syntax:**
`while condition:`
 `print( 'Running' )`

```
x = 0
while x < 3:
    print(x)
    x = x + 1
print(x)
```

while Loops (1): Kombination mit User-Input

- Möglichkeit: Ausführen bis spezieller Input
- Kontrolle durch Bedingungen oder „Flags“ (= Boolean-Variable)

```
user_input = ''  
while user_input != 'quit':  
    user_input = input('What are you saying?')  
    print(user_input)
```

while Loops (2): Endlosschleifen (Infinite Loops)

- Programm läuft unendlich
 - `while True`
- Abbruchbedingung durch `break`

```
while True:  
    user_input = input('What are you saying?')  
    print(user_input)  
    if user_input == 'quit':  
        break
```

Loops: Keywords zum Schleifen- und Programmablauf

- `pass`
 - „leerer“ Command, tut nichts
- `break`
 - Schleifenabbruch
 - Sofortiges Beenden der „innersten Schleife“
- `continue`
 - Sprung zu nächster Iteration
 - Abbruch aktueller Durchlauf „innerster Schleife“